

Rapport TP1

Recommendation system: Collaborative Filtering item-item top-N

Nom et prenom : AGBESSI John Ulrich

Filiere : Genie logiciel

Ecole : IFRI-UAC

Depot GitHub :

<https://github.com/John271200/TP1Recommendation-system.git>

Application Streamlit : <https://tp1recommendation-system.streamlit.app/>

1. Introduction

Ce projet porte sur la réalisation d'un système de recommandation basé sur le filtrage collaboratif item-item. L'objectif est de proposer à un utilisateur une liste top-N d'items qu'il n'a pas encore notés, en utilisant les notes données par l'ensemble des utilisateurs.

Le projet contient aussi une interface Streamlit permettant de tester facilement le système depuis un navigateur.

2. Objectif du projet

L'objectif principal est de construire une application capable de lire un fichier de notes utilisateur-item, calculer les similarités entre les items, prédire les scores des items non notés par un utilisateur et afficher les meilleures recommandations top-N.

3. Données utilisées

Le projet utilise deux fichiers CSV. Le fichier `ratings.csv` contient les notes données par les utilisateurs. Le fichier `items.csv` contient les noms des items. Dans ce TP, les items représentent des films.

`user_id,item_id,rating`

`U1,I1,5`

`U1,I2,4`

`item_id,title`

`I1,Inception`

4. Methode utilisee

La methode utilisee est le filtrage collaboratif item-item. Elle consiste a comparer les items entre eux a partir des notes donnees par les utilisateurs.

Le programme construit une structure utilisateur-item pour connaitre les notes de chaque utilisateur, puis une structure item-utilisateur pour connaitre les utilisateurs ayant note chaque item.

Pour chaque paire d'items, le programme construit deux vecteurs de notes. Une note manquante est consideree comme 0. La similarite utilisee est la similarite cosinus.

$$\text{similarite}(A, B) = \text{produit_scalaire}(A, B) / (\text{norme}(A) * \text{norme}(B))$$

Pour un utilisateur cible, le programme ignore les items deja notes. Pour chaque item candidat, il calcule un score predit avec une moyenne ponderee par les similarites.

$$\text{score}(\text{item}) = \text{somme}(\text{similarite} * \text{note}) / \text{somme}(\text{abs}(\text{similarite}))$$

5. Implementation

Le fichier recommender.py contient l'implementation de l'algorithme de recommandation. Le fichier app.py contient l'interface Streamlit. Les fichiers ratings.csv et items.csv contiennent les donnees. Le fichier requirements.txt contient la dependance Streamlit.

L'application Streamlit permet de choisir un utilisateur, choisir le nombre de recommandations, afficher les notes deja donnees et afficher les similarites item-item.

6. Resultats obtenus

La commande console suivante a ete testee :

```
python recommender.py --user U1 --top-n 3 --show-similarities
```

Resultat obtenu :

Top similarites item-item

John Wick <-> Mad Max Fury Road : 0.986

The Matrix <-> Mad Max Fury Road : 0.977

The Matrix <-> John Wick : 0.962

Inception <-> The Martian : 0.898

Interstellar <-> The Martian : 0.828

Recommandations top-3 pour U1

1. Gravity (I7) - score predit : 4.27

2. Blade Runner 2049 (I8) - score predit : 2.69

3. John Wick (I4) - score predict : 1.94

Pour l'utilisateur U1, les films déjà notés sont exclus des recommandations. Le système propose donc seulement des items non notés par cet utilisateur.

7. Déploiement

Le projet est disponible sur GitHub :

<https://github.com/John271200/TP1Recommendation-system.git>

L'application Streamlit est disponible ici :

<https://tp1recommendation-system.streamlit.app/>

Le professeur peut accéder au dépôt avec l'adresse :

johnaoga@gmail.com

8. Conclusion

Ce TP a permis de mettre en place un système de recommandation top-N avec filtrage collaboratif item-item. L'approche est simple, interprétable et efficace pour comprendre les bases des systèmes de recommandation.

L'ajout de Streamlit rend le projet plus accessible, car l'utilisateur peut tester les recommandations directement dans une interface web.